EN3160 Assignment 3: Neural Networks

H.D.J.D.Samaranayaka | **230563H** | GitHub: EN3160-Neural-Networks | PDF: `230563H_a02.pdf`

The hidden first cell applies print formatting. Run All uses saved results (`RUN_TRAINING = False`); export the HTML as A4 PDF with backgrounds enabled.

Three ten-epoch experiments: a dense CIFAR-10 classifier, two MNIST CNNs, and ImageNet-pretrained ResNet-18 transfer learning on Hymenoptera.

Question 1

Method

Flatten each 32 by 32 RGB image to 3,072 inputs; use a 100-unit sigmoid hidden layer and 10 logits, cross-entropy loss, SGD (learning rate 0.01, momentum 0.9), and 10 epochs.

In [3]:
```
import torch
from torch import nn

class DenseNetCIFAR(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(3 * 32 * 32, 100)
        self.output = nn.Linear(100, 10)

    def forward(self, x):
        x = x.reshape(x.size(0), -1)
        return self.output(torch.sigmoid(self.hidden(x)))

# Training objective and optimizer
criterion = nn.CrossEntropyLoss()
model = DenseNetCIFAR()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
```

In [4]:
```
if RUN_TRAINING:
    run_module("src.dense_network.train", "--epochs", "10", "--data", str(PROJECT_ROOT / "data"))
else:
    print("Using saved CIFAR-10 results. Set RUN_TRAINING = True to retrain.")

with (PROJECT_ROOT / "reports/cifar10_dense_history.json").open(encoding="utf-8") as f:
    history = json.load(f)

print(f"Final train accuracy: {history['train_accuracy'][-1]:.2%}")
print(f"Final test accuracy:  {history['test_accuracy'][-1]:.2%}")
```

```
Using saved CIFAR-10 results. Set RUN_TRAINING = True to retrain.
Final train accuracy: 52.08%
Final test accuracy:  48.67%
```

Question 2

Method

Train LeNet-5 and a three-convolution CNN with batch normalization and dropout on the same MNIST split for 10 epochs. The result cell reports final train/test metrics and displays per-epoch loss/accuracy plots and test confusion matrices for both models.

In [5]:
```
from torch import nn

class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 6, 5), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(6, 16, 5), nn.ReLU(), nn.MaxPool2d(2))
        self.classifier = nn.Sequential(
            nn.Flatten(), nn.Linear(16 * 4 * 4, 120), nn.ReLU(),
            nn.Linear(120, 84), nn.ReLU(), nn.Linear(84, 10))
    def forward(self, x):
        return self.classifier(self.features(x))

class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.network = nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(),
            nn.Conv2d(16, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2), nn.Dropout(0.25), nn.Flatten(),
            nn.Linear(64 * 7 * 7, 128), nn.ReLU(), nn.Dropout(0.5),
            nn.Linear(128, 10))
```

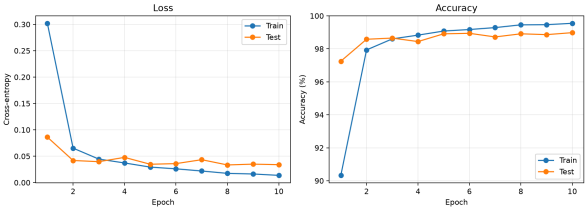
```
    def forward(self, x):
        return self.network(x)
```

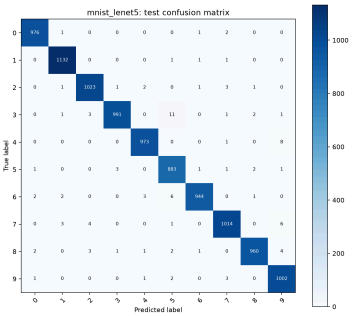
In [6]:

```
if RUN_TRAINING:
    run_module("src.mnist.train", "--epochs", "10", "--data", str(PROJECT_ROOT / "data"))
else:
    print("Using saved MNIST results. Set RUN_TRAINING = True to retrain.")

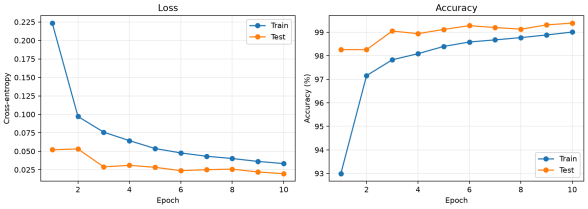
for model_name in ("lenet5", "small_cnn"):
    with (PROJECT_ROOT / f"reports/mnist_{model_name}_history.json").open(encoding="utf-8") as f:
        history = json.load(f)
    print(f"{model_name}: final train accuracy={history['train_accuracy'][-1]:.2%}, "
          f"test accuracy={history['test_accuracy'][-1]:.2%}, "
          f"test loss={history['test_loss'][-1]:.4f}")
    display(Image(filename=str(PROJECT_ROOT / f"outputs/figures/mnist_{model_name}_learning_curves.png")))
    display(Image(filename=str(PROJECT_ROOT / f"outputs/figures/mnist_{model_name}_confusion_matrix.png")))
```

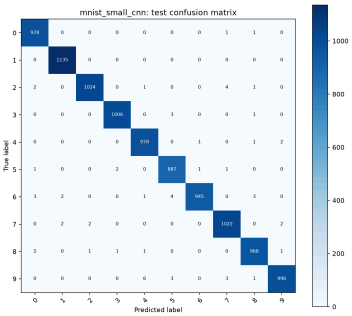
```
Using saved MNIST results. Set RUN_TRAINING = True to retrain.
lenet5: final train accuracy=99.55%, test accuracy=98.98%, test loss=0.0339
```






```
small_cnn: final train accuracy=99.01%, test accuracy=99.39%, test loss=0.0196
```





Question 3

Method

Use ImageNet1K-pretrained ResNet-18 and replace its final layer for ants and bees. Compare full fine-tuning against a frozen feature extractor with only the replacement classifier trained; both run for 10 epochs.

In [7]:
```python
from torch import nn
from torchvision import models

def make_resnet18(num_classes, freeze_backbone):
    model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
    if freeze_backbone:
        for parameter in model.parameters():
            parameter.requires_grad = False
    model.fc = nn.Linear(model.fc.in_features, num_classes)
    return model

# Fine-tune all layers with freeze_backbone=False;
# train only model.fc with freeze_backbone=True.
```

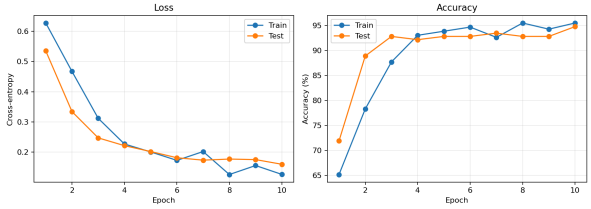
In [8]:
```python
if RUN_TRAINING:
    run_module("src.transfer_learning.train", "--epochs", "10", "--data", str(PROJECT_ROOT / "data"))
else:
    print("Using saved transfer-learning results. Set RUN_TRAINING = True to retrain.")

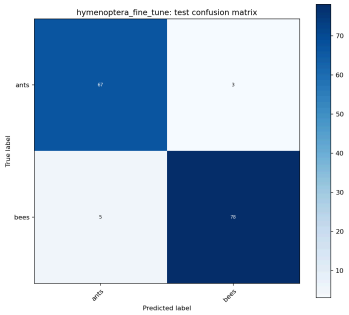
with (PROJECT_ROOT / "reports/hymenoptera_comparison.json").open(encoding="utf-8") as f:
    print(json.dumps(json.load(f), indent=2))

for model_name, strategy in (("fine_tune", "Fine-tuning"), ("feature_extractor", "Frozen feature extractor")):
    print(f"{strategy}: learning curves and validation confusion matrix")
    display(Image(filename=str(PROJECT_ROOT / f"outputs/figures/hymenoptera_{model_name}_learning_curves.png")))
    display(Image(filename=str(PROJECT_ROOT / f"outputs/figures/hymenoptera_{model_name}_confusion_matrix.png")))
```

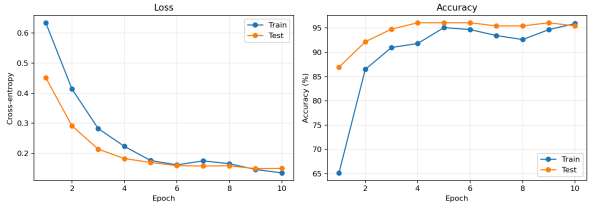
```
Using saved transfer-learning results. Set RUN_TRAINING = True to retrain.
{
  "fine_tune": {
    "test_accuracy": 0.9477124183006536,
    "test_loss": 0.15941916805466796,
    "trainable_parameters": 11177538
  },
  "feature_extractor": {
    "test_accuracy": 0.954248366013072,
    "test_loss": 0.15046505951413922,
    "trainable_parameters": 1026
  }
}
```

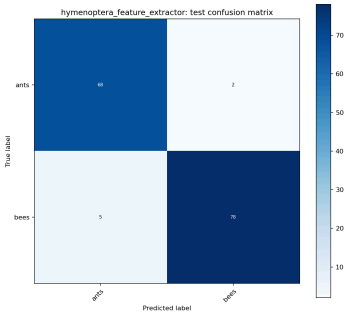
Fine-tuning: learning curves and validation confusion matrix





Frozen feature extractor: learning curves and validation confusion matrix





Results and discussion

Question 1 — CIFAR-10 dense network

Epochs	Train accuracy	Test accuracy
10	52.08%	48.67%

The train/test gap is 3.41 percentage points, so the run shows a modest generalization gap. Accuracy improves across training, but the final scores are limited for this 10-class image task: flattening discards spatial layout, while a single hidden layer has less capacity to learn local image features.

Question 2 — MNIST CNN comparison

Model	Train accuracy	Test accuracy	Test loss
LeNet-5	99.55%	98.98%	0.0339
Three-layer CNN	99.01%	99.39%	0.0196

Both models' training loss falls across epochs, and test accuracy settles above 98%. LeNet-5 has a 0.57 percentage-point train/test gap; its most frequent error is digit 3 classified as 5 (11 cases). The three-layer CNN reaches 99.39% test accuracy, 0.41 points above LeNet-5, with lower test loss (0.0196 vs. 0.0339). Its test accuracy slightly exceeds its training accuracy because dropout is active during training but disabled for evaluation. Both confusion matrices are strongly diagonal; the CNN most often confuses 9 with 4 (6 cases).

Question 3 — ResNet-18 transfer learning

Setup	Trainable parameters	Train accuracy	Validation accuracy	Validation loss
Fine-tuning	11,177,538	95.49%	94.77%	0.1594
Frozen feature extractor	1,026	95.90%	95.42%	0.1505

The frozen feature extractor leads this run by 0.65 percentage points in validation accuracy and uses far fewer trainable parameters. Both validation curves improve over the run; the frozen extractor starts higher (86.93% vs. 71.90%) and finishes slightly higher. Its final train/validation gap is 0.48 percentage points, compared with 0.72 for fine-tuning, suggesting the pretrained features already transfer well on this small dataset. The validation split contains 153 images, so one additional correct prediction changes accuracy by about 0.65 points.

The CNNs exploit image structure and perform strongly on MNIST. ImageNet pretraining supports high Hymenoptera validation accuracy; freezing the backbone is more parameter-efficient and slightly better on this split. Results are specific to this run and split.

Shared training function

The experiment scripts use this common training/evaluation loop: cross-entropy loss, SGD with momentum, backpropagation, and per-epoch train/test loss and accuracy. It works with each model shown above.

In [9]:

```
import torch
from torch import nn

def fit(model, train_loader, test_loader, epochs, learning_rate, device):
    model.to(device); criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate, momentum=0.9)
    history = dict(train_loss=[], train_accuracy=[], test_loss=[], test_accuracy=[])
    for _ in range(epochs):
        model.train(); loss_sum = correct = count = 0
        for x, y in train_loader:
            x, y = x.to(device), y.to(device); optimizer.zero_grad(set_to_none=True)
            logits = model(x); loss = criterion(logits, y)
            loss.backward(); optimizer.step()
            loss_sum += loss.item() * y.size(0)
            correct += logits.argmax(1).eq(y).sum().item(); count += y.size(0)
        history["train_loss"].append(loss_sum/count); history["train_accuracy"].append(correct/count)
        model.eval(); loss_sum = correct = count = 0
        with torch.inference_mode():
            for x, y in test_loader:
                x, y = x.to(device), y.to(device); logits = model(x)
                loss_sum += criterion(logits, y).item() * y.size(0)
                correct += logits.argmax(1).eq(y).sum().item(); count += y.size(0)
        history["test_loss"].append(loss_sum/count); history["test_accuracy"].append(correct/count)
    return history
```